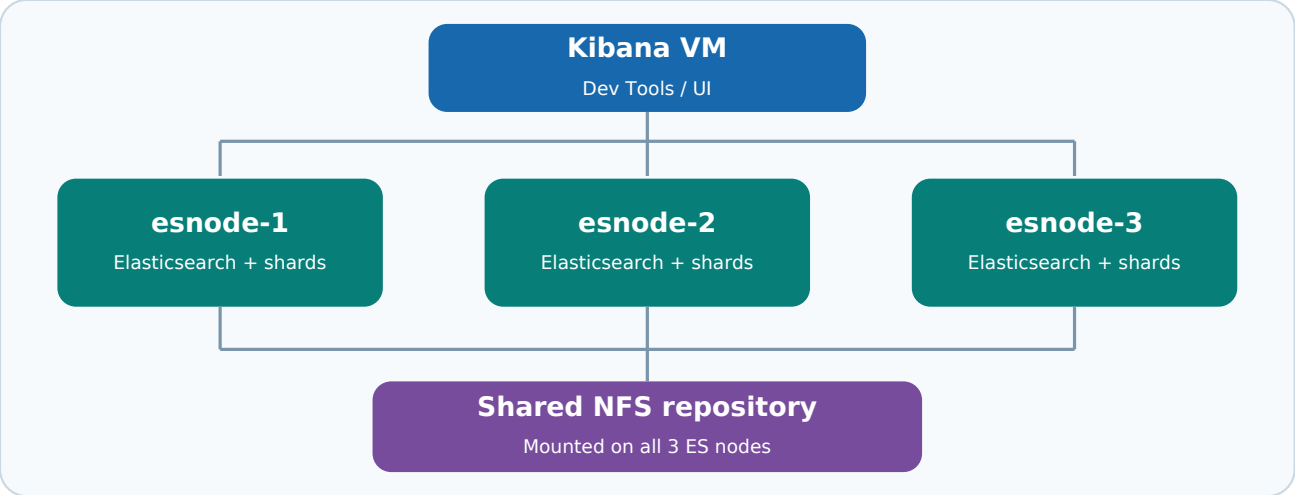
OPERATIONS LAB MANUAL

Elasticsearch Snapshot Lifecycle, Restore & Controlled Node-Failure Recovery

Ubuntu | 3 Elasticsearch nodes + 1 Kibana node | Hands-on instructor and participant edition



Lab design
Create a shared NFS snapshot repository, back up a test index, restore it under a new name, schedule SLM snapshots, simulate a single-node outage, and verify automatic shard recovery.

Parameter	Lab value
Environment	esnode-1, esnode-2, esnode-3; Kibana on a separate Ubuntu VM
Storage	Shared NFS export; separate from Elasticsearch data directories
Tools	Ubuntu terminal, sudo, nano, Kibana Dev Tools
Safety scope	Test index only; controlled single-node stop/start
Prerequisites	Working TLS-enabled Elasticsearch cluster, Kibana access, adequate disk and RAM

Prepared 06 November 2026 | Guided workshop

1. Goals, architecture and safe operating rules

Learning objectives: distinguish replicas from backups; register and verify a shared repository; capture and inspect a snapshot; safely restore a single index; automate snapshot lifecycle management (SLM); observe primary and replica recovery after one node fails; collect evidence of success.

1.1 Snapshot vs replica vs node recovery

Mechanism	Protects against	Does not protect against
Replica shard	Failure of one node containing a shard copy	Accidental deletion replicated across copies; loss of all copies
Snapshot	Logical deletion, data loss, rebuild on compatible cluster	Live zero-RPO failover; local config files
Master election	Loss of current master when voting quorum remains	Loss of quorum or index data

1.2 Preflight safety gates

- Use a disposable training index named **snapshot-lab-index**. Never run delete or overwrite commands against production data.
- Confirm all 3 Elasticsearch nodes are in the same cluster. For the master-election drill, confirm 3 master-eligible voting nodes before stopping the elected master.
- Stop only one Elasticsearch service at a time. Wait until it rejoins and the cluster stabilizes before starting a different failure test.
- Do not change cluster UUID, reset node metadata, force an unsafe bootstrap or allocate stale primaries in this workshop.
- Keep snapshots on separate persistent storage. A folder copied to each node is **not** a shared repository.
- This lab does not back up **elasticsearch.yml**, TLS certificates, secure settings or Kibana server config files. Back up those separately.

1.3 Assign your lab parameters

Variable	Fill in during the lab
KIBANA_VM_IP / NFS_SERVER_IP	_____
ES_NODE_1_IP / ES_NODE_2_IP / ES_NODE_3_IP	_____
LAB_SUBNET_CIDR	_____
Repository mount path	/mnt/es-snapshots
Repository name	lab_fs
Snapshot name	lab-snapshot-01

Important
The commands assume Debian/Ubuntu **apt** packages and **systemd**. They do not use dpkg. Commands are separated by execution location: Kibana Dev Tools, Kibana VM terminal, and each Elasticsearch VM.

2. Lab A - Verify existing cluster and roles

STEP A1 Open Kibana -> Dev Tools -> Console

Use Kibana global search if the navigation varies by Kibana version. Confirm the cluster is responsive.

Kibana Dev Tools / Console

```
GET /
GET _cluster/health?pretty
GET _cat/nodes?v&h=name,ip,node.role,master
GET _cat/master?v
GET _cat/shards?v
```

Expected: number_of_nodes = 3; each expected esnode is listed. Green indicates all primaries and replicas assigned. Yellow indicates all primaries assigned but at least one replica unassigned; red indicates an unavailable primary.

STEP A2 Record master and shard placement

Kibana Dev Tools / Console

```
GET _cluster/state?filter_path=metadata.cluster_coordination.last_committed_config
GET _cat/allocation?v
GET _cluster/settings?include_defaults=false
```

Interpretation: the asterisk in **_cat/nodes** identifies the current master. Do not assume node 1 must be the elected master. A healthy 3-node cluster might still be yellow if disk or shard allocation constraints prevent replicas.

STEP A3 Check Ubuntu Elasticsearch service on each ES VM

Ubuntu terminal - run on esnode-1, -2, -3

```
hostname
sudo systemctl status elasticsearch --no-pager
sudo journalctl -u elasticsearch -n 25 --no-pager
```

Checkpoint A
If the cluster does not contain all three nodes, resolve discovery/TLS/network issues before any failure drill. Confirm working search and adequate disk space first.

Troubleshooting hints

Symptom	First checks
Kibana Dev Tools not reachable	systemctl status kibana; Kibana -> Elasticsearch TLS trust and URL
Only 2 ES nodes listed	Elasticsearch service, logs, discovery.seed_hosts, firewall and TLS
Cluster yellow/red	_cat/shards, _cat/allocation, _cluster/allocation/explain

3. Lab B - Build shared NFS snapshot storage

For this **training lab only**, the separate Kibana VM can host NFS if it has adequate spare disk. For production, use independent, durable NFS or a supported cloud/object-storage repository. Perform NFS steps only if you do not already have a working shared export.

STEP B1 On the Kibana VM, install and prepare the NFS server

Ubuntu terminal - Kibana VM

```
sudo apt update
sudo apt install -y nfs-kernel-server
sudo mkdir -p /srv/es-snapshots
sudo systemctl enable --now nfs-kernel-server
```

STEP B2 Find Elasticsearch service numeric UID/GID

Ubuntu terminal - esnode-1

```
id elasticsearch
```

Record the numeric uid and gid. They must be consistent for NFS permissions. If ES nodes have different IDs, correct the identity/permission design before proceeding.

STEP B3 Set ownership of NFS export to matching numeric UID/GID

Ubuntu terminal - Kibana VM

```
sudo chown <ES_UID>:<ES_GID> /srv/es-snapshots
sudo chmod 0770 /srv/es-snapshots
sudo nano /etc/exports
```

Add one export line, replacing the CIDR with your lab ES subnet:

nano - /etc/exports

```
/srv/es-snapshots <LAB_SUBNET_CIDR>(rw,sync,no_subtree_check,root_squash)
```

Ubuntu terminal - Kibana VM

```
sudo exportfs -ra
sudo exportfs -v
sudo systemctl status nfs-kernel-server --no-pager
```

Allow NFS traffic only from the trusted lab subnet using your environment firewall rules. Do not expose NFS to the public Internet.

STEP B4 On each Elasticsearch VM, install the NFS client and mount export

Ubuntu terminal - repeat on esnode-1, -2, -3

```
sudo apt update
sudo apt install -y nfs-common
sudo mkdir -p /mnt/es-snapshots
sudo mount -t nfs <NFS_SERVER_IP>:/srv/es-snapshots \
  /mnt/es-snapshots
findmnt /mnt/es-snapshots
df -h /mnt/es-snapshots
```

B5 - Confirm shared ownership and persistence

The mount output must show the **same NFS server/export** on all three ES nodes. Test read/write access using the Elasticsearch account and plan an appropriate persistent /etc/fstab entry before node reboot.

Ubuntu terminal - repeat on all 3 ES nodes

```
sudo -u elasticsearch test -w /mnt/es-snapshots && echo WRITABLE
sudo -u elasticsearch test -r /mnt/es-snapshots && echo READABLE
```

Expected result
findmnt identifies the same NFS export on every ES node; both read and write tests print success. If permission tests fail, fix numeric UID/GID mapping, NFS access, export options and Linux permissions before registering the repository.

4. Lab C - Configure and verify snapshot repository

STEP C1 Set path.repo on every Elasticsearch VM

Ubuntu terminal - esnode-1, -2, -3

```
sudo nano /etc/elasticsearch/elasticsearch.yml
```

Add the following YAML once at top level; avoid duplicate path.repo keys:

nano - elasticsearch.yml

```
path.repo: ["/mnt/es-snapshots"]
```

STEP C2 Perform a controlled rolling restart to apply path.repo

On esnode-1, restart and wait until it rejoins; then esnode-2; then esnode-3. Never restart them all together. For routine production rolling restarts, follow Elastic's full runbook and consider temporarily suspending replica allocation; do not change allocation in this introductory lab.

Ubuntu terminal - one ES VM at a time

```
sudo systemctl restart elasticsearch
sudo systemctl is-active elasticsearch
```

Kibana Dev Tools - after each restart

```
GET _cat/nodes?v
GET _cluster/health?pretty
```

STEP C3 Register repository using Kibana Dev Tools

Kibana Dev Tools / Console

```
PUT _snapshot/lab_fs
{
  "type": "fs",
  "settings": {
    "location": "/mnt/es-snapshots/lab_fs",
    "compress": true
  }
}
```

The repository directory is managed by Elasticsearch. Do not manually edit, move or clean individual repository files.

STEP C4 Verify the repository across participating nodes

Kibana Dev Tools / Console

```
POST _snapshot/lab_fs/_verify
GET _snapshot/lab_fs
```

Expected: a **nodes** object listing participating nodes in the verification response, and a registered fs repository in the GET response.

Optional Kibana UI verification

Navigate to **Stack Management -> Snapshot and Restore -> Repositories**. Find **lab_fs**. Menu names may vary slightly with Kibana version.

Checkpoint C
Repository registration may fail with repository_exception or a verification error if NFS is not mounted on **all master and data nodes**, path.repo is missing, or the Elasticsearch service cannot write to shared storage.

5. Lab D - Create a manual snapshot of test data

STEP D1 Create a simple test index with one replica

Kibana Dev Tools / Console

```
PUT snapshot-lab-index
{
  "settings": {
    "number_of_shards": 1,
    "number_of_replicas": 1
  }
}
PUT snapshot-lab-index/_doc/1
{
  "message": "Before snapshot",
  "phase": "baseline"
}
POST snapshot-lab-index/_refresh
```

STEP D2 Verify current contents and shard placement

Kibana Dev Tools / Console

```
GET snapshot-lab-index/_count
GET snapshot-lab-index/_search
GET _cat/shards/snapshot-lab-index?v&h=index,shard,prirep,state,node
```

Expected: count = 1, primary shard **p** and replica shard **r** assigned to different data nodes. If replica is UNASSIGNED, resolve that before the failover exercise.

STEP D3 Take the snapshot and wait until it finishes

Kibana Dev Tools / Console

```
PUT _snapshot/lab_fs/lab-snapshot-01?wait_for_completion=true
{
  "indices": "snapshot-lab-index",
  "include_global_state": false
}
```

STEP D4 Inspect metadata and result

Kibana Dev Tools / Console

```
GET _snapshot/lab_fs/lab-snapshot-01
GET _snapshot/lab_fs/lab-snapshot-01/_status
GET _snapshot/lab_fs/_all
```

Expected: snapshot state = **SUCCESS**; failures = 0; includes snapshot-lab-index. A successful start does not itself prove successful completion, so inspect final state.

Evidence item	Record actual value
Repository verification nodes	_____
Snapshot name / state	_____
Number of shards copied / failed	_____
Index count before snapshot	_____

Snapshot scope
This particular snapshot excludes global cluster state and Elastic feature states. It is suitable for the disposable index exercise, **not a complete disaster-recovery strategy**.

6. Lab E - Safe restore to a renamed index

STEP E1 Add a document after the backup

Kibana Dev Tools / Console

```
PUT snapshot-lab-index/_doc/2
{
  "message": "After snapshot",
  "phase": "later-change"
}
POST snapshot-lab-index/_refresh
GET snapshot-lab-index/_count
```

Expected: 2 documents in the original index. The earlier snapshot should contain only document 1.

STEP E2 Restore into a NEW index; keep source data untouched

Kibana Dev Tools / Console

```
POST _snapshot/lab_fs/lab-snapshot-01/_restore?wait_for_completion=true
{
  "indices": "snapshot-lab-index",
  "include_global_state": false,
  "include_aliases": false,
  "rename_pattern": "snapshot-lab-index",
  "rename_replacement": "restored-snapshot-lab-index"
}
```

STEP E3 Validate original and restored data independently

Kibana Dev Tools / Console

```
GET snapshot-lab-index/_count
GET restored-snapshot-lab-index/_count
GET restored-snapshot-lab-index/_search
GET _cat/indices/*snapshot-lab-index?v
```

Object	Expected count	Meaning
snapshot-lab-index	2	Original remains unchanged
restored-snapshot-lab-index	1	Backup state, before second insert

E4 - Optional restore-state checks

Kibana Dev Tools / Console

```
GET _cat/recovery/restored-snapshot-lab-index?v
GET _cluster/health/restored-snapshot-lab-index?pretty
```

The restore API may finish initiating a restore before all replica work is complete; inspect health and recovery to confirm allocation.

Safe practice
Do not delete snapshot-lab-index to simulate corruption in the first exercise. Restoring to a separate index makes the outcome easy to compare without overwriting existing data.

7. Lab F - Automate backup via Snapshot Lifecycle Management

STEP F1 Create an SLM policy for the lab index

Kibana Dev Tools / Console

```
PUT _slm/policy/lab-daily
{
  "schedule": "0 0 2 * * ?",
  "name": "<lab-daily-{now/d}>",
  "repository": "lab_fs",
  "config": {
    "indices": "snapshot-lab-index",
    "include_global_state": false
  },
  "retention": {
    "expire_after": "7d",
    "min_count": 1,
    "max_count": 7
  }
}
```

Time: 02:00 UTC = 07:30 India Standard Time. SLM schedules use the nodes' system clock; keep clocks synchronized. The policy appends a unique identifier to scheduled snapshot names.

STEP F2 Run policy now to test it (schedule remains unchanged)

Kibana Dev Tools / Console

```
POST _slm/policy/lab-daily/_execute
GET _slm/policy/lab-daily
GET _slm/stats
```

STEP F3 Verify that the resulting snapshot has finished

Kibana Dev Tools / Console

```
GET _snapshot/lab_fs/_all
GET _snapshot/lab_fs/_current
GET _slm/status
```

An SLM execution response means a snapshot was requested; look for its eventual **SUCCESS** state in repository listing.

STEP F4 Review retention behavior

The policy retains between 1 and 7 policy-created snapshots while applying the 7-day age limit. Its retention rules do **not** automatically govern your manually created lab-snapshot-01. Retention is a separate scheduled operation.

Kibana Dev Tools / Console

```
POST _slm/_execute_retention
GET _slm/stats
```

Production consideration
Use a separate reviewed policy covering required business indices, data streams, system feature states, ILM and templates. Protect repository credentials and test full restore on a separate compatible cluster. This example intentionally limits its scope to one training index.

8. Lab G - Controlled one-node outage and recovery

G0 - Entry criteria and baseline

Ensure cluster health is green and primary+replica of the test index are assigned to different nodes. Identify the current master. Select a **non-master** Elasticsearch node that holds either test-index shard copy; if none meets the criteria, adjust the lab instead of forcing a failure.

Kibana Dev Tools / Console

```
GET _cat/master?v
GET _cat/nodes?v&h=name,node.role,master
GET _cat/shards/snapshot-lab-index?v&h=index,shard,prirep,state,node
GET _cluster/health/snapshot-lab-index?pretty
```

STEP G1 Capture baseline evidence

Before outage	Record
Master node	_____
Primary location	_____
Replica location	_____
Cluster health / number of nodes	_____

STEP G2 Stop ONE chosen Elasticsearch service

Ubuntu terminal - CHOSEN non-master ES VM ONLY

```
sudo systemctl stop elasticsearch
sudo systemctl is-active elasticsearch
```

Expected service state is inactive. **Do not stop Kibana, NFS, or another ES node.**

STEP G3 Observe cluster and search continuity

Kibana Dev Tools / Console

```
GET _cat/nodes?v
GET _cluster/health?pretty
GET _cat/shards/snapshot-lab-index?v
GET _cat/recovery?v
GET snapshot-lab-index/_search
```

Expected: node count drops from 3 to 2. Search still succeeds if a usable shard copy survives. If the primary was on the stopped node, Elasticsearch can promote its available replica. Health may become yellow or transiently red during detection, depending on placement and allocation.

G4 - Record what happened

Observation during outage	Record
Stopped node	_____
Remaining node count	_____
New primary location	_____
Health status at first observation	_____
Search succeeded?	YES / NO

9. Lab G continued - Return node and verify recovery

STEP G5 Restart the stopped node

Ubuntu terminal - SAME ES VM that was stopped

```
sudo systemctl start elasticsearch
sudo systemctl is-active elasticsearch
sudo journalctl -u elasticsearch -n 30 --no-pager
```

STEP G6 Inspect shard recovery and health

Kibana Dev Tools / Console

```
GET _cat/nodes?v
GET _cluster/health?wait_for_status=green&timeout=120s
GET _cat/shards/snapshot-lab-index?v
GET _cat/recovery?v
GET snapshot-lab-index/_count
```

Expected: all three nodes rejoin; the training index primary and replica become assigned; count remains 2. Green might take time due to delayed allocation, disk pressure, or other indices; investigate instead of repeatedly restarting nodes.

STEP G7 Check allocation explanation if not green

Kibana Dev Tools / Console

```
GET _cluster/allocation/explain
GET _cat/allocation?v
GET _cluster/settings?include_defaults=true&filter_path=**.allocation
```

Inspect the actual reasons returned, especially disk watermarks, node roles, unassigned replicas, allocation filters and insufficient nodes.

Optional master-failure exercise - advanced

Only after full recovery and verifying that the voting configuration includes 3 healthy master-eligible members, repeat a one-node outage on the current elected master. Remaining voting nodes should elect another master if a majority stays online. Restore the stopped service before any further test.

Kibana Dev Tools / Console

```
GET _cat/nodes?v&h=name,node.role,master
GET _cat/master?v
GET _cluster/state?filter_path=metadata.cluster_coordination.last_committed_config
```

Do NOT force elections
Elasticsearch automatically manages voting configuration. Never use unsafe-bootstrap, detach-cluster, data-path deletion or force-stale-primary techniques as routine recovery steps in this workshop.

Failure recovery vs snapshot recovery: decision point

Failure	Correct first response
One node offline; shards still available	Restore/restart node; allow automatic replica recovery
Accidentally deleted test index	Restore index from verified snapshot (rename first)
All voting quorum lost	Bring back existing voting nodes; do not form a new cluster
No available shard copy	Investigate original node/copy; restore backed-up data if necessary

10. Troubleshooting playbook

Issue / likely symptom	Diagnostic and remedy
Repository not found / location not permitted	Check path.repo in elasticsearch.yml on every master+data node; apply rolling restart; confirm repo path.
repository_verification_exception	Check NFS mount on every ES node; identical shared export and permissions; execute repository _verify.
Permission denied writing repository	Check id elasticsearch, server export directory ownership, numeric UID/GID, mount and exportfs -v.
NFS mount absent after reboot	Check persistent /etc/fstab entry, network readiness, NFS server and findmnt.
Snapshot PARTIAL / FAILED	Look at _snapshot/lab_fs/<name> and _status; verify primary shards, disk, repository availability.
Snapshot named already exists	Use a new unique snapshot name; never overwrite a repository object manually.
Restore reports index already exists	Use a unique rename_replacement. No production index deletion is needed.
Restored count differs	Check snapshot state, which index was snapshotted, request scope, and post-snapshot writes.
SLM policy not running	GET _slm/status, _slm/policy/lab-daily, _slm/stats; check UTC cron and privileges.
Node has not rejoined	systemctl status, journalctl, disk pressure, ES discovery/TLS and cluster logs.
Cluster stays yellow	Review _cat/shards and _cluster/allocation/explain; do not infer data loss from yellow alone.
Cluster red / no master	Check quorum and failed node restoration; do not start unrelated reset procedures.

Security & durability
Keep NFS restricted to lab machines, protect snapshots from unauthorized modification, verify backups regularly, and take separate protected backups of Elasticsearch configuration, TLS and secure settings. Avoid shared-filesystem corruption by allowing only Elasticsearch to write repository internals.

Ubuntu quick reference

Ubuntu terminal - affected ES node

```
sudo systemctl status elasticsearch --no-pager
sudo journalctl -u elasticsearch -n 50 --no-pager
findmnt /mnt/es-snapshots
df -h /mnt/es-snapshots
id elasticsearch
```

11. Completion checklist and learner evidence

Use this page as the lab assessment sheet. Capture Kibana Console responses or terminal screenshots as evidence.

- ☐ Three ES nodes appear in _cat/nodes and the cluster is operational.
- ☐ NFS export is accessible from all ES nodes at the same mount path.
- ☐ elasticsearch.yml includes path.repo on every required ES node.
- ☐ Repository lab_fs registered and _verify successful.
- ☐ Test index created with one primary and one replica.
- ☐ Manual snapshot lab-snapshot-01 has final state SUCCESS.
- ☐ Original index count 2; restored index count 1.
- ☐ SLM policy lab-daily created and manual execution verified.
- ☐ Exactly one non-master Elasticsearch node stopped; search still works.
- ☐ Stopped node restarted and rejoined; shards and expected data verified.
- ☐ Troubleshooting and safety gates followed; no production index changed.

Evidence screenshot / output	File / location / observation
Cluster baseline	_____
Repository _verify	_____
Manual snapshot SUCCESS	_____
Restored count comparison	_____
SLM policy status	_____
Outage recovery / shard movement	_____
Final _cat/nodes and health	_____

Instructor sign-off: _____ **Date:** _____

12. Key concepts and official references

Concept recap

- **Snapshot repository:** storage registered with Elasticsearch that preserves backup contents and metadata.
- **Manual snapshot:** on-demand backup of selected indices and optional cluster state.
- **SLM:** policy that schedules snapshots and independently applies policy-scoped retention.
- **Replica promotion:** available replica becomes primary when an active primary disappears.
- **Quorum:** more than half of the current voting configuration must be able to participate.
- **Controlled recovery:** stop one node, observe cluster changes, restart it, and verify node and shard reintegration.

Official Elastic documentation

Snapshot and restore overview
https://www.elastic.co/docs/deploy-manage/tools/snapshot-and-restore

Shared file system repository
https://www.elastic.co/docs/deploy-manage/tools/snapshot-and-restore/shared-file-system-repository

Create, monitor and delete snapshots / SLM
https://www.elastic.co/docs/deploy-manage/tools/snapshot-and-restore/create-snapshots

Restore a snapshot
https://www.elastic.co/docs/deploy-manage/tools/snapshot-and-restore/restore-snapshot

Rolling restart procedure
https://www.elastic.co/docs/deploy-manage/maintenance/start-stop-services/full-cluster-restart-rolling-restart-procedures

Quorum-based decision making
https://www.elastic.co/docs/deploy-manage/distributed-architecture/discovery-cluster-formation/modules-discovery-quorums

Cluster fault detection
https://www.elastic.co/docs/deploy-manage/distributed-architecture/discovery-cluster-formation/cluster-fault-detection

Scope note
Examples are illustrative and were not executed against your actual VMs. Replace placeholders with your verified node addresses and existing NFS parameters. Exact UI labels and API permissions depend on your installed Elasticsearch/Kibana version.